

Documento de Decisiones de Diseño: Sistema de Autenticación EcoMarket

Este documento detalla las justificaciones técnicas y de seguridad detrás de las decisiones arquitectónicas implementadas en el módulo de autenticación de la plataforma EcoMarket para el nivel de Auditoría de Seguridad.

Decisión de Diseño	Justificación Técnica	Riesgo Mitigado
Almacenamiento en RAM	Uso de variables volátiles de proceso.	Exfiltración de credenciales físicas.
Margen de 300s	Ventana de 5 minutos para renovación.	Interrupción por latencia de red.
Locking (Singleton)	threading.Lock() con doble comprobación.	Estampida de peticiones (Thundering Herd).
Logout por 401 en Refresh	Cierre inmediato de sesión.	Bucles infinitos y baneo de IP.

1. Estrategia de Almacenamiento: Memoria Volátil (RAM)

Con lo que platicamos con las preguntas el almacenar tanto el access_token como el refresh_token estrictamente en la memoria RAM del proceso activo de Python. Esta decisión prioriza la seguridad contra ataques de exfiltración de datos locales, ya que al no persistir los tokens en archivos de configuración o bases de datos sin cifrar, reducimos drásticamente la superficie de ataque ante intrusos que obtengan acceso al sistema de archivos.

Escenario de fallo: En el caso de una terminación forzada del proceso (comando kill -9), los tokens se borran permanentemente. Aunque esto obliga al usuario a re-autenticarse, es una medida de seguridad necesaria para evitar el riesgo de "tokens huérfanos" en disco que, si carecieran de los permisos correctos (como un permiso 600 en Linux), podrían ser leídos por otros usuarios del servidor para suplantar la identidad del cliente EcoMarket.

2. Margen de Refresh Proactivo: 300 Segundos (5

Minutos)

Con lo dicho en las preguntas yo me acordaba que eran 30 segundos pero el margen de renovación se estableció en 300 segundos antes de la expiración real (exp) del JWT. Esta ventana de 5 minutos proporciona un colchón de resiliencia necesario para absorber fluctuaciones en la latencia de red o micro-caídas en el servicio de autenticación de la plataforma, permitiendo reintentos silenciosos en segundo plano.

Escenario de fallo: Si el servidor de EcoMarket experimenta una carga alta y tarda 45 segundos en procesar la solicitud de renovación, un margen insuficiente (como 30 segundos) causaría que el token original expirara antes de recibir el nuevo. Con 5 minutos, el sistema tiene tiempo suficiente para manejar retrasos del backend o errores de red temporales sin que el usuario perciba interrupciones en su flujo de trabajo.

3. Manejo de Concurrencia: Refresh Singleton con `threading.Lock`

Implementamos el patrón de *Double-Checked Locking* utilizando `threading.Lock()` para asegurar que el proceso de renovación sea un "Singleton" funcional. Esto garantiza que, independientemente de cuántos hilos detecten la expiración simultáneamente, solo la primera petición realice la llamada de red al endpoint de `/refresh`, mientras las demás esperan el resultado actualizado.

Escenario de fallo: Ante una "Estampida de Peticiones" (Thundering Herd), donde múltiples módulos de la aplicación solicitan datos concurrentemente al expirar el token, el candado evita disparar múltiples llamadas de refresh. Sin este control, se saturaría la infraestructura de EcoMarket y se correría el riesgo de invalidar tokens previos debido a las políticas de rotación de secretos que implementan los servidores OAuth2 modernos.

4. Comportamiento ante Refresh Fallido: Logout Inmediato

En la lógica del interceptor incluye una condición crítica: si la respuesta del endpoint de renovación devuelve un código 401, el sistema detiene cualquier reintento y ejecuta una limpieza total mediante `logout()`. Esta decisión protege la integridad del cliente al reconocer que las credenciales de renovación ya no son válidas o han sido revocadas por el administrador.

Escenario de fallo: Esta medida previene de forma efectiva el "Bucle Infinito de la Muerte", donde una aplicación intenta refrescar un token inválido miles de veces por segundo. Sin el cierre de sesión inmediato, la aplicación podría ser identificada como un ataque de Denegación de Servicio (DoS) por los firewalls de EcoMarket, resultando en el baneo permanente de la dirección IP del usuario legítimo.